applied sciences

MDPI

Article

HeteroGCL: A Heterogeneous Graph Contrastive Learning Framework for Scalable and Sustainable Cryptocurrency AML

Jiaying Chen [1], Jingyi Liu [2], Yiwen Liang [1,*] and Mengjie Zhou [3]

[1] SC Johnson Graduate School of Management, Cornell University, Ithaca, NY 14853, USA; jc2744@cornell.edu
[2] College of Computing and Information Science, Cornell University, Ithaca, NY 14853, USA; jl3758@cornell.edu
[3] Department of Computer Science, University of Bristol, Bristol BS8 1SS, UK
* Correspondence: yl2947@cornell.edu

Abstract

Anti-money laundering (AML) in cryptocurrency networks presents significant challenges due to complex transactional relationships, severe class imbalance, and limited labeled data, which severely constrain the scalability and label efficiency of existing AML systems. Traditional machine learning approaches treat transactions independently and fail to capture the intricate network structures inherent in money laundering schemes. To address these limitations, we propose HeteroGCL, a heterogeneous graph contrastive learning framework for scalable and sustainable cryptocurrency AML. Our approach models cryptocurrency transactions as a heterogeneous graph with multiple node and edge types and integrates a heterogeneous graph attention network with a graph contrastive learning module. By leveraging unlabeled data through topology-aware and attribute-aware graph augmentations, HeteroGCL mitigates label scarcity while enabling scalable and label-efficient AML model training while reducing reliance on costly manual annotation. Extensive experiments on the Elliptic dataset demonstrate that HeteroGCL achieves superior performance over state-of-the-art baselines, achieving an F1-score of 0.824 and an AUC of 0.912, with a 4.7% improvement in F1-score compared to the CARE-GNN baseline. The results indicate that the proposed framework effectively captures complex money laundering patterns while supporting scalable deployment of AML systems and improving the economic and operational sustainability of blockchain AML infrastructures.

Keywords: anti-money laundering; heterogeneous graph neural networks; contrastive learning; cryptocurrency; fraud detection

Academic Editor: Gianluca Lax

Received: 13 February 2026
Revised: 11 March 2026
Accepted: 12 March 2026
Published: 16 March 2026

1. Introduction

The rapid proliferation of cryptocurrencies has revolutionized financial transactions, offering unprecedented levels of decentralization, anonymity, and cross-border accessibility. However, these same characteristics have inadvertently created fertile ground for illicit activities, particularly money laundering. According to recent estimates, cryptocurrency-based money laundering exceeded $8.6 billion in 2021, representing a significant threat to global financial security and the sustainable development of legitimate cryptocurrency markets [1]. The pseudonymous nature of blockchain transactions, combined with the absence of centralized oversight, poses substantial challenges for regulatory compliance, law enforcement agencies, and the long-term sustainability of blockchain-based financial ecosystems [2,3].

Traditional anti-money laundering (AML) systems rely predominantly on rule-based detection mechanisms and supervised machine learning models that analyze individual transactions in isolation [4,5]. These approaches typically extract hand-crafted features such as transaction amounts, frequencies, and temporal patterns, feeding them into classifiers like Random Forests, Support Vector Machines, or gradient boosting models [6,7]; while effective for detecting known patterns, these methods suffer from critical limitations: (1) they fail to capture the relational structures and collective behaviors inherent in money laundering schemes, which often involve coordinated networks of accounts executing layered transactions [5]; (2) they struggle with the severe class imbalance problem, where illicit transactions constitute less than 2% of total volume [5]; and (3) they require extensive labeled data for training, which is scarce and expensive to obtain in the AML domain, hindering the scalable and cost-effective deployment of AML systems [8].

Money laundering operations typically manifest as organized networks or "rings" where multiple accounts collaborate to obfuscate the origin of illicit funds through complex transaction chains [4,9]. This network-based nature suggests that graph-based approaches could provide superior detection capabilities by explicitly modeling the relational dependencies between entities. Recent advances in graph neural networks (GNNs) have demonstrated remarkable success in learning representations from graph-structured data across various domains, including social network analysis [10], recommendation systems [11], and fraud detection [9,12]. However, cryptocurrency transaction networks exhibit inherent heterogeneity, comprising diverse node types (e.g., transactions, addresses, temporal clusters) and edge types (e.g., input flows, output flows, co-occurrence relationships) with distinct semantic meanings [5]. Homogeneous GNN architectures that treat all nodes and edges uniformly fail to leverage this rich structural information, potentially degrading detection performance.

Heterogeneous graph neural networks (HGNNs) address this limitation by explicitly modeling different node and edge types through type-specific transformations and aggregation mechanisms [13,14]. The heterogeneous graph attention network (HGAT) [13] and its variants have shown promising results in various applications by employing hierarchical attention mechanisms to capture both node-level and semantic-level importance. Despite these advances, supervised HGNNs still face the fundamental challenge of label scarcity in AML scenarios, where obtaining ground-truth labels requires extensive manual investigation and domain expertise [5].

Contrastive learning has emerged as a powerful paradigm for self-supervised representation learning, achieving state-of-the-art performance in computer vision [15,16] and natural language processing [17]. The core principle is to learn representations by maximizing agreement between differently augmented views of the same instance while minimizing agreement with other instances. Recent works have successfully adapted contrastive learning to graph-structured data [18–20], demonstrating its effectiveness in learning robust node embeddings without requiring labeled data. Graph contrastive learning (GCL) employs graph augmentation strategies such as node dropping, edge perturbation, and attribute masking to generate multiple views of the input graph [18]. The InfoNCE loss [21] serves as a widely adopted objective function, treating augmented versions of the same node as positive pairs while treating all other nodes as negative samples.

However, existing GCL methods primarily focus on homogeneous graphs and may not adequately preserve the semantic information in heterogeneous networks when applying standard augmentation strategies [22]. Naively dropping nodes or edges in a heterogeneous graph could destroy critical type-specific patterns essential for money laundering detection. Furthermore, the design of effective augmentation strategies for AML tasks requires careful

consideration of domain knowledge to ensure that augmented graphs maintain realistic transaction patterns [8].

In this paper, we propose HeteroGCL, a novel framework that synergistically integrates heterogeneous graph neural networks with contrastive learning for anti-money laundering in cryptocurrency transactions. Our approach addresses the aforementioned challenges through the following key contributions:

- We propose HeteroGCL, a heterogeneous graph contrastive learning framework tailored to cryptocurrency AML. Rather than introducing a generic heterogeneous contrastive objective, our key conceptual contribution is to formulate AML detection as a heterogeneous relational representation learning problem in which fund-flow, temporal-association, and behavioral co-occurrence relations are modeled jointly within one domain-informed graph schema.
- We introduce domain-aware graph augmentation strategies for heterogeneous transaction graphs. In contrast to prior heterogeneous graph contrastive learning methods developed for generic recommendation or meta-path settings, our augmentations are designed to preserve transaction semantics that are critical in AML, especially informative fund-flow structure and discriminative transaction attributes.
- We design a unified learning objective for label-scarce and class-imbalanced AML that couples self-supervised contrastive learning with supervised classification and focal re-weighting. This formulation is intended to improve label efficiency in AML settings where annotations are expensive and illicit samples are rare, rather than to claim a universally optimal heterogeneous contrastive objective.
- Extensive experiments on the Elliptic dataset demonstrate that the proposed framework achieves superior performance compared with state-of-the-art baselines.

The remainder of this paper is organized as follows. Section 2 reviews related work in anti-money laundering, graph neural networks, and contrastive learning. Section 3 introduces necessary preliminaries and problem formulation. Section 4 presents the detailed methodology of HeteroGCL, including heterogeneous graph construction, the enhanced HGAT architecture, and the contrastive learning framework. Section 5 reports extensive experimental results and analysis. Finally, Section 6 concludes this paper with discussions on limitations and future research directions.

Sustainability Perspective: In this work, the notion of sustainability refers to the efficient and responsible use of computational and annotation resources in AML systems. Specifically, sustainable AML solutions should reduce reliance on costly labeled data, maintain computational efficiency when scaling to large transaction networks, and remain adaptable to evolving financial ecosystems. The proposed HeteroGCL framework contributes to these goals by leveraging contrastive learning to improve label efficiency, employing scalable heterogeneous graph modeling, and supporting robust detection in large blockchain transaction graphs.

2. Related Works

In this section, we review the existing literature related to our work from three perspectives: anti-money laundering techniques, graph neural networks for fraud detection, and contrastive learning on graphs.

2.1. Anti-Money Laundering Techniques

Traditional AML approaches predominantly rely on rule-based systems and statistical anomaly detection methods. Financial institutions typically employ transaction monitoring systems that flag suspicious activities based on predefined rules, such as transactions exceeding certain thresholds or exhibiting unusual patterns [23]. However, these rule-

based systems suffer from high false-positive rates and can be easily circumvented by adaptive adversaries [24].

With the advancement of machine learning, supervised learning techniques have been widely adopted for AML tasks. Jullum et al. [25] applied Random Forests and XGBoost to detect suspicious transactions in banking systems, achieving improved accuracy over traditional methods. Pourhabibi et al. [26] conducted a comprehensive survey on fraud detection techniques in banking, highlighting the effectiveness of ensemble methods and deep neural networks. However, these approaches treat transactions as independent instances, ignoring the network structure inherent in money laundering schemes, which limits their effectiveness and long-term operational viability in real-world deployment.

In the context of cryptocurrency-based money laundering, several studies have explored blockchain-specific features. Bartoletti et al. [27] performed data mining on Bitcoin transactions to identify patterns associated with illicit activities. Ostapowicz and Żbikowski [28] utilized machine learning classifiers combined with behavioral features extracted from blockchain data. Despite these efforts, the extreme class imbalance (typically less than 2% illicit samples) and limited labeled data remain significant challenges [7].

Network analysis approaches have shown promise in capturing the relational nature of money laundering, offering more scalable and resource-efficient detection strategies. Colladon and Remondi [24] employed social network analysis techniques to detect anomalous patterns in financial transaction networks. Savage et al. [4] proposed a method to identify money laundering groups by analyzing network topology features. These studies demonstrate that incorporating network structure improves detection performance and supports sustainable AML practices, motivating the adoption of graph-based deep learning methods.

2.2. Graph Neural Networks for Fraud Detection

Graph neural networks have revolutionized the processing of graph-structured data by enabling end-to-end learning of node and graph representations. The seminal work by Kipf and Welling [29] introduced graph convolutional networks (GCNs), which generalize convolutional operations to irregular graph structures. Subsequently, graph attention networks (GATs) [30] enhanced GCNs by incorporating attention mechanisms to weight the importance of neighboring nodes dynamically.

In the fraud detection domain, several GNN-based approaches have been proposed. Wang et al. [9] developed a semi-supervised graph attentive network for financial fraud detection, demonstrating superior performance over traditional methods. Liu et al. [12] addressed the class imbalance problem in fraud detection by proposing a GNN-based framework that selectively samples neighbors based on their informativeness. Cheng et al. [31] applied graph convolutional networks to detect fraudulent transactions in e-commerce platforms, showing that GNNs effectively capture collaborative fraud patterns.

For cryptocurrency AML specifically, Weber et al. [5] pioneered the application of GCNs to the Elliptic dataset, demonstrating that graph-based features significantly outperform hand-crafted features. Alarab et al. [6] evaluated various GNN architectures for Bitcoin AML, finding that deeper networks with residual connections achieve better performance. Pareja et al. [32] proposed EvolveGCN to handle temporal dynamics in evolving graphs, which is relevant for tracking money laundering activities over time.

However, most existing GNN approaches for AML treat transaction graphs as homogeneous, where all nodes and edges are of the same type. This simplification overlooks the rich semantic information present in cryptocurrency transaction networks, where different types of relationships (e.g., input flows, output flows, temporal co-occurrences) carry dis-

tinct meanings. Heterogeneous graph neural networks address this limitation by explicitly modeling type-specific transformations.

Wang et al. [13] proposed the heterogeneous graph attention network (HGat), which employs node-level and semantic-level attention to aggregate information across different node and edge types. Hu et al. [14] introduced the Heterogeneous Graph Transformer (HGT), incorporating relative temporal encoding and heterogeneous mutual attention. Zhang et al. [33] developed a meta-path-based approach for heterogeneous graph representation learning. Recent work such as [34] demonstrates how incorporating domain knowledge through expert-mixture architectures can significantly improve both reasoning capability and interpretability in complex financial decision-making tasks. Similarly, Ref. [35] shows that hybrid Transformer–GNN models can achieve strong performance while maintaining interpretability in high-stakes financial classification problems, highlighting the value of combining structural representation learning with explainable AI techniques; while these methods have shown success in domains such as recommendation systems and academic citation networks, their application to AML remains underexplored, particularly in combination with self-supervised learning techniques.

2.3. Contrastive Learning on Graphs

Contrastive learning has emerged as a powerful self-supervised paradigm that learns representations by contrasting positive pairs against negative samples. In computer vision, methods like SimCLR [15] and MoCo [16] have achieved remarkable success by applying various augmentation strategies to create multiple views of images. The InfoNCE loss [21] serves as the foundational objective function, maximizing mutual information between representations of positive pairs.

Extending contrastive learning to graph-structured data presents unique challenges due to the irregular structure and discrete nature of graphs. You et al. [18] proposed GraphCL, which employs graph augmentation techniques including node dropping, edge perturbation, attribute masking, and subgraph sampling. Zhu et al. [19] introduced Deep Graph Infomax (DGI), which contrasts node representations with graph-level summaries. Hassani and Khasahmadi [20] developed MVGRL, a multi-view framework that generates graph views through diffusion and sampling.

Recent works have explored graph contrastive learning in specific application contexts. Qiu et al. [36] proposed GCC for pre-training graph neural networks across multiple datasets. Xu et al. [37] introduced InfoGCL, which adaptively selects informative augmentations based on information theory principles. Suresh et al. [38] applied adversarial graph augmentation to improve robustness. However, these methods primarily focus on homogeneous graphs and may not adequately preserve semantic information when applied to heterogeneous networks. Existing graph contrastive learning methods mainly focus on homogeneous graphs or generic heterogeneous settings without considering domain-specific relational semantics.

For heterogeneous graphs, recent efforts have begun to adapt contrastive learning principles. Ren et al. [22] proposed a heterogeneous graph contrastive learning framework for recommendation systems, employing type-aware augmentation strategies. Zhang et al. [39] developed a self-supervised method that contrasts different meta-path-based views of heterogeneous graphs. Despite these advances, the application of heterogeneous graph contrastive learning to financial fraud detection, particularly in addressing label scarcity and class imbalance, remains largely unexplored.

Our work differs from existing heterogeneous graph contrastive learning methods in both scope and intent. Prior heterogeneous contrastive approaches are typically developed for generic heterogeneous graphs, recommendation settings, or meta-path-based represen-

tation learning. By contrast, HeteroGCL is motivated by a specific AML representation problem: different relation types in cryptocurrency transactions correspond to different investigative signals, and naive heterogeneous augmentations can easily distort those signals. Accordingly, the conceptual contribution of HeteroGCL lies in aligning graph construction, augmentation design, and the training objective with AML-specific semantics and data constraints. In this sense, the method should be viewed as a domain-specialized heterogeneous graph contrastive framework for label-scarce AML, rather than as a claim of a new universally general heterogeneous contrastive paradigm.

3. Preliminaries

In this section, we introduce the fundamental concepts and notations used throughout this paper, including heterogeneous graphs, graph neural networks, and contrastive learning.

3.1. Heterogeneous Graph

A heterogeneous graph is defined as a graph with multiple types of nodes and edges, which can be formally represented as follows.

Definition 1 (Heterogeneous Graph). *A heterogeneous graph is defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A}, \mathcal{R})$, where $\mathcal{V}$ denotes the set of nodes and $\mathcal{E}$ denotes the set of edges. Each node $v \in \mathcal{V}$ and each edge $e \in \mathcal{E}$ are associated with their type mapping functions $\phi(v) : \mathcal{V} \to \mathcal{A}$ and $\psi(e) : \mathcal{E} \to \mathcal{R}$, where $\mathcal{A}$ and $\mathcal{R}$ denote the sets of node types and edge types, respectively. When $|\mathcal{A}| + |\mathcal{R}| > 2$, the graph is heterogeneous.*

In the context of cryptocurrency transaction networks, nodes can represent different entities such as transactions, addresses, or temporal clusters, while edges can represent various relationships such as input flows, output flows, or temporal co-occurrences. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^d$, where d is the dimensionality of the feature space.

3.2. Graph Neural Networks

Graph neural networks operate by iteratively aggregating information from a node's local neighborhood to learn node representations. The general message-passing framework can be expressed as follows:

$$\mathbf{h}_i^{(l+1)} = \text{AGGREGATE}^{(l)}\left(\left\{\mathbf{h}_j^{(l)} : j \in \mathcal{N}(i)\right\}\right), \tag{1}$$

where $\mathbf{h}_i^{(l)}$ denotes the representation of node i at layer l, and $\mathcal{N}(i)$ represents the set of neighbors of node i. The aggregation function can take various forms, such as mean pooling in GCNs [29] or attention-based weighting in GATs [30].

For heterogeneous graphs, the aggregation process must account for different node and edge types. The heterogeneous graph attention network (HGAT) [13] introduces type-specific transformations and hierarchical attention mechanisms to handle this heterogeneity, which we build upon in our framework.

3.3. Contrastive Learning

Contrastive learning aims to learn representations by maximizing agreement between positive pairs while minimizing agreement with negative samples. Given an input sample

x, two augmented views $\tilde{x}_i$ and $\tilde{x}_j$ are generated through data augmentation. The InfoNCE loss [21] is commonly used as the objective function:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{I}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}, \tag{2}$$

where $\mathbf{z}_i$ and $\mathbf{z}_j$ are the representations of the two views, $\text{sim}(\cdot, \cdot)$ denotes a similarity function (typically cosine similarity), τ is a temperature parameter, and N is the batch size. The numerator encourages similar representations for positive pairs, while the denominator pushes apart representations of different samples.

In graph contrastive learning, augmentation strategies such as node dropping, edge perturbation, and feature masking are applied to generate different views of the graph [18]. The challenge lies in designing augmentations that preserve semantic information while introducing sufficient variation for effective contrastive learning.

3.4. Problem Formulation

Given a heterogeneous cryptocurrency transaction graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A}, \mathcal{R})$ with node features $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_{|\mathcal{V}|}\}$, where only a small subset of nodes $\mathcal{V}_L \subset \mathcal{V}$ are labeled with binary labels $\mathbf{Y}_L = \{y_i \in \{0, 1\} : v_i \in \mathcal{V}_L\}$ indicating illicit (1) or licit (0) transactions, our goal is to learn a function $f : \mathcal{V} \to \{0, 1\}$ that accurately predicts the labels of unlabeled nodes $\mathcal{V}_U = \mathcal{V} \setminus \mathcal{V}_L$.

The key challenges are: (1) severe class imbalance with $|\{v_i : y_i = 1\}| \ll |\{v_i : y_i = 0\}|$, (2) limited labeled data with $|\mathcal{V}_L| \ll |\mathcal{V}|$, and (3) complex relational patterns involving multiple node and edge types. Our HeteroGCL framework addresses these challenges by combining heterogeneous graph neural networks with contrastive learning to leverage both labeled and unlabeled data effectively.

4. Methodology

In this section, we present the detailed methodology of our HeteroGCL framework. We begin by describing the construction of heterogeneous transaction graphs from cryptocurrency data, followed by our enhanced heterogeneous graph attention network architecture. We then introduce the graph contrastive learning module with domain-aware augmentation strategies, and finally present the unified training objective that combines supervised and self-supervised learning. The design of HeteroGCL is guided by the observation that cryptocurrency money laundering patterns often emerge from the interaction of multiple relational signals, including fund flow, temporal coordination, and behavioral similarity. Therefore, our framework integrates heterogeneous relational modeling with contrastive representation learning to capture these complementary signals.

4.1. Heterogeneous Transaction Graph Construction

Cryptocurrency transaction networks inherently exhibit heterogeneous structures due to the diverse entities and relationships involved, while existing approaches often simplify these networks into homogeneous graphs; such simplification discards valuable semantic information that can be crucial for detecting sophisticated money laundering schemes and ensuring the sustainability of blockchain-based financial systems.

We construct a heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A}, \mathcal{R})$ where the node set $\mathcal{V}$ consists of two types: transaction nodes and temporal cluster nodes. Transaction nodes represent individual Bitcoin transactions, each characterized by features including transaction amount, number of inputs/outputs, transaction fee, and timestamp-derived features. Temporal cluster nodes aggregate transactions occurring within the same time window, capturing

temporal co-occurrence patterns that are often exploited in coordinated money laundering activities. Formally, we define the node type set as $\mathcal{A} = \{\text{Transaction}, \text{Temporal}\}$.

We choose transaction nodes and temporal cluster nodes as the two node types for the following reasons. First, the Elliptic dataset is organized at the transaction level, where each node corresponds to a transaction with associated features, while address- or wallet-level identity/ownership information is not explicitly provided. Incorporating address or wallet nodes would therefore require additional heuristic entity resolution or clustering, which may introduce noise and reduce reproducibility. Second, transaction-level modeling offers a more scalable representation for large blockchain networks: adding address/wallet nodes can substantially increase the number of nodes and edges, leading to higher computational and memory costs. Our design thus balances expressiveness and scalability. Nevertheless, extending HeteroGCL to richer heterogeneous schemas that incorporate addresses/wallets and smart-contract entities is an important direction for future work.

Instead of adopting a fully dynamic graph neural network, we introduce temporal cluster nodes as a lightweight mechanism for incorporating temporal information into the heterogeneous graph. This design is motivated by the discrete time-step organization of the Elliptic dataset and by the practical objective of keeping the framework computationally manageable on large transaction graphs. Specifically, temporal cluster nodes, temporal-association edges, short-window co-occurrence edges, and timestamp-derived node features together allow the model to encode coarse temporal context and short-term coordination patterns while remaining compatible with heterogeneous contrastive learning. At the same time, this design does not model event-level temporal evolution explicitly: it cannot represent fine-grained transaction ordering, variable inter-event intervals, or long-range temporal dependencies in the same way as continuous-time or dynamic GNNs. We therefore view the current temporal design as a scalable approximation of temporal dynamics rather than a full temporal modeling solution, and richer dynamic formulations remain an important direction for future work.

The edge set $\mathcal{E}$ captures three distinct types of relationships. Transaction flow edges connect consecutive transactions where the output of one transaction serves as the input to another, representing the actual movement of funds through the network. These edges are directed and carry semantic meaning about fund provenance. Temporal association edges connect transactions to their corresponding temporal clusters, establishing membership relationships. Co-occurrence edges link transactions that share common attributes such as similar timestamps or overlapping address sets, which may indicate coordinated behavior. The edge type set is therefore $\mathcal{R} = \{\text{Flow}, \text{Temporal-Association}, \text{Co-occurrence}\}$.

This heterogeneous formulation enables our model to distinguish between different relational semantics. For instance, fund flow relationships carry stronger implications for money laundering detection than mere temporal proximity, and our architecture leverages this distinction through type-specific processing.

4.2. Enhanced Heterogeneous Graph Attention Network

Building upon the HGAT architecture, we design an enhanced heterogeneous graph attention network that effectively captures multi-relational patterns in cryptocurrency transaction networks. The core challenge is to aggregate information from heterogeneous neighbors while preserving type-specific semantic meanings.

Given a target node v_i of type $\phi(v_i) \in \mathcal{A}$, we first project its features into a type-specific latent space. For each node type $a \in \mathcal{A}$, we define a type-specific transformation

matrix $\mathbf{W}_a \in \mathbb{R}^{d' \times d}$, where d is the input feature dimension and d' is the hidden dimension. The initial node representation is computed as follows:

$$\mathbf{h}_i^{(0)} = \mathbf{W}_{\phi(v_i)}\mathbf{x}_i + \mathbf{b}_{\phi(v_i)}, \tag{3}$$

where $\mathbf{b}_{\phi(v_i)}$ is a type-specific bias vector. This type-specific transformation ensures that nodes of different types are projected into semantically meaningful subspaces.

For information aggregation across heterogeneous neighbors, we employ a hierarchical attention mechanism operating at both the node level and the edge-type level. At the node level, for each edge type $r \in \mathcal{R}$, we compute attention coefficients that quantify the importance of neighboring nodes connected via edges of type r. Given a target node v_i and its neighbor v_j connected by edge type r, the node-level attention coefficient is

$$\alpha_{ij}^r = \frac{\exp\left(\text{LeakyReLU}\left(\mathbf{a}_r^T[\mathbf{W}_r\mathbf{h}_i^{(l)} \| \mathbf{W}_r\mathbf{h}_j^{(l)}]\right)\right)}{\sum_{k \in \mathcal{N}_i^r} \exp\left(\text{LeakyReLU}\left(\mathbf{a}_r^T[\mathbf{W}_r\mathbf{h}_i^{(l)} \| \mathbf{W}_r\mathbf{h}_k^{(l)}]\right)\right)}, \tag{4}$$

where $\mathbf{W}_r \in \mathbb{R}^{d' \times d'}$ is an edge-type-specific transformation matrix, $\mathbf{a}_r \in \mathbb{R}^{2d'}$ is an attention vector for edge type r, $\|$ denotes concatenation, and $\mathcal{N}_i^r$ represents the set of neighbors of node i connected via edges of type r. The LeakyReLU activation introduces non-linearity while avoiding the vanishing gradient problem.

After computing node-level attention, we aggregate representations from neighbors of the same edge type:

$$\mathbf{h}_i^{r,(l)} = \sigma\left(\sum_{j \in \mathcal{N}_i^r} \alpha_{ij}^r \mathbf{W}_r \mathbf{h}_j^{(l)}\right), \tag{5}$$

where σ is a non-linear activation function. This produces an intermediate representation $\mathbf{h}_i^{r,(l)}$ for each edge type r, capturing information from homogeneous neighborhoods.

The edge-type-level attention mechanism then aggregates these intermediate representations across different edge types. Different edge types contribute differently to the final representation based on their semantic relevance to the target node. We compute edge-type attention coefficients as follows:

$$\beta_i^r = \frac{\exp\left(\frac{1}{|\mathcal{N}_i^r|}\sum_{j \in \mathcal{N}_i^r} \mathbf{q}^T \tanh\left(\mathbf{W}_s \mathbf{h}_i^{r,(l)} + \mathbf{b}_s\right)\right)}{\sum_{r' \in \mathcal{R}_i} \exp\left(\frac{1}{|\mathcal{N}_i^{r'}|}\sum_{j \in \mathcal{N}_i^{r'}} \mathbf{q}^T \tanh\left(\mathbf{W}_s \mathbf{h}_i^{r',(l)} + \mathbf{b}_s\right)\right)}, \tag{6}$$

where $\mathbf{q} \in \mathbb{R}^{d'}$ is a semantic-level attention vector, $\mathbf{W}_s \in \mathbb{R}^{d' \times d'}$ is a transformation matrix, and $\mathcal{R}_i$ denotes the set of edge types connected to node i. The averaging over neighbors $|\mathcal{N}_i^r|$ normalizes the contribution of each edge type by its neighborhood size, preventing dominant edge types from overshadowing sparse but informative relationships.

The final node representation at layer $l+1$ is obtained by aggregating across all edge types:

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\sum_{r \in \mathcal{R}_i} \beta_i^r \mathbf{h}_i^{r,(l)}\right). \tag{7}$$

This hierarchical attention mechanism allows the model to adaptively focus on the most relevant neighbors and edge types for each node. In the context of money laundering detection, this is particularly valuable as illicit transactions often exhibit distinctive patterns in specific relationship types. For example, rapid fund flows through multiple intermediary accounts may be more indicative of layering behavior than temporal co-occurrence patterns.

We apply this architecture for L layers, with residual connections to facilitate gradient flow:

$$\mathbf{h}_i^{(l+1)} = \mathbf{h}_i^{(l+1)} + \mathbf{h}_i^{(l)}. \tag{8}$$

The final node representation $\mathbf{h}_i^{(L)}$ serves as input to both the classification head and the contrastive learning module.

4.3. Graph Contrastive Learning with Domain-Aware Augmentation

The severe scarcity of labeled data in AML scenarios motivates the integration of self-supervised learning to enable scalable and label-efficient AML solutions. However, naively applying standard graph augmentation strategies can destroy critical structural and semantic information necessary for money laundering detection. We design domain-aware augmentation strategies that preserve transaction semantics while introducing sufficient variation for effective contrastive learning, supporting resource-efficient and sustainable model development.

Our contrastive learning framework generates two augmented views of the input graph through carefully designed augmentation operations. The key principle is to preserve the fundamental characteristics that define money laundering patterns while introducing controlled perturbations that encourage the model to learn robust and invariant representations.

For the first augmentation strategy, we employ topology-aware edge perturbation. Rather than randomly dropping edges, we selectively perturb edges based on their importance to the overall graph structure. We compute edge importance scores using a combination of centrality measures and type-specific weights. For each edge e_{ij} of type r, the importance score is

$$s_{ij}^r = w_r \cdot \big(\mathrm{BC}(e_{ij}) + \lambda \cdot \mathrm{ECC}(v_i, v_j)\big), \tag{9}$$

where w_r is a learnable weight for edge type r, $\mathrm{BC}(e_{ij})$ denotes edge betweenness centrality, $\mathrm{ECC}(v_i, v_j)$ represents the edge clustering coefficient, and λ is a balancing parameter. We then sample edges for removal with probability inversely proportional to their importance scores, ensuring that critical transaction flows are preserved. Specifically, the removal probability for edge e_{ij} is

$$p_{\text{remove}}(e_{ij}) = 1 - \frac{s_{ij}^r}{\max_{e \in \mathcal{E}} s_e}. \tag{10}$$

This topology-aware perturbation maintains the backbone structure of transaction networks while introducing local variations that prevent overfitting to specific graph topologies. The topology-aware edge perturbation relies on structural statistics such as edge betweenness centrality and edge clustering coefficients. To maintain scalability, these measures are computed as a preprocessing step before training rather than during each training iteration. Furthermore, for large graphs we employ approximate or sampling-based algorithms for estimating edge betweenness, which significantly reduces computational overhead compared with exact computation. Since these statistics are used only to guide the probability of edge perturbation, approximate values are sufficient in practice. As a result, this component introduces negligible overhead relative to the overall graph neural network training process.

For the second augmentation strategy, we apply attribute-aware feature masking. Transaction features often exhibit high dimensionality with varying degrees of relevance. We identify robust feature subsets through mutual information analysis and selectively mask less

informative features. For node v_i with feature vector $\mathbf{x}_i = [x_{i1}, x_{i2}, \ldots, x_{id}]$, we compute the mutual information between each feature dimension and the downstream task:

$$\mathrm{MI}(x_{\cdot k}, y) = \sum_{x_k, y} p(x_k, y) \log \frac{p(x_k, y)}{p(x_k)p(y)}, \tag{11}$$

where $x_{\cdot k}$ denotes the k-th feature dimension computed over labeled nodes in the training split only. To prevent any information leakage under the temporal split protocol, we compute $MI(x_{\cdot k}, y)$ only on labeled transaction nodes in the training period (time steps 1–34), denoted as $\mathcal{V}_L^{\text{train}}$. No validation/test labels (time steps 35–49) are used in this computation, and unlabeled nodes are excluded from MI estimation. For continuous-valued features, we discretize $x_{\cdot k}$ into B bins using training-set quantiles and estimate $p(x_k, y)$ and $p(x_k)$ empirically from $\mathcal{V}_L^{\text{train}}$ (with Laplace smoothing), after which the MI scores are computed once per run and kept fixed during training. Features with lower mutual information scores are masked with higher probability:

$$p_{\text{mask}}(x_{\cdot k}) = 1 - \frac{\mathrm{MI}(x_{\cdot k}, y)}{\max_{k'} \mathrm{MI}(x_{\cdot k'}, y)}. \tag{12}$$

This ensures that critical features such as transaction amounts and network-based features remain largely intact while introducing sufficient noise for effective contrastive learning.

Given the original graph $\mathcal{G}$ and two augmentation functions t_1 and t_2 implementing the aforementioned strategies, we generate two augmented views $\tilde{\mathcal{G}}_1 = t_1(\mathcal{G})$ and $\tilde{\mathcal{G}}_2 = t_2(\mathcal{G})$. These augmented graphs are then processed through the same heterogeneous graph attention network to obtain node representations $\{\tilde{\mathbf{h}}_i^{(1)}\}$ and $\{\tilde{\mathbf{h}}_i^{(2)}\}$.

To produce the final contrastive representations, we apply a projection head consisting of a multi-layer perceptron:

$$\mathbf{z}_i^{(1)} = g_{\text{proj}}\left(\tilde{\mathbf{h}}_i^{(1)}\right), \quad \mathbf{z}_i^{(2)} = g_{\text{proj}}\left(\tilde{\mathbf{h}}_i^{(2)}\right), \tag{13}$$

where g_{proj} is a two-layer MLP with ReLU activation. The projection head maps the node representations into a space where the contrastive loss is computed, following the practice established in recent contrastive learning literature that shows such projection improves representation quality.

The contrastive loss encourages representations of the same node from different augmented views to be similar while pushing apart representations of different nodes. We employ the InfoNCE loss formulated as follows:

$$\mathcal{L}_{\text{contrast}} = -\frac{1}{|\mathcal{V}|} \sum_{i=1}^{|\mathcal{V}|} \log \frac{\exp\left(\mathrm{sim}\left(\mathbf{z}_i^{(1)}, \mathbf{z}_i^{(2)}\right)/\tau\right)}{\sum_{k=1}^{|\mathcal{V}|} \mathbb{I}_{[k \neq i]} \exp\left(\mathrm{sim}\left(\mathbf{z}_i^{(1)}, \mathbf{z}_k^{(2)}\right)/\tau\right)}, \tag{14}$$

where $\mathrm{sim}(\mathbf{u}, \mathbf{v}) = \mathbf{u}^T\mathbf{v}/(\|\mathbf{u}\|\|\mathbf{v}\|)$ is the cosine similarity function, and τ is a temperature parameter controlling the concentration of the distribution. This loss function maximizes the mutual information between representations of positive pairs while treating all other nodes as negative samples.

4.4. Unified Training Objective

To leverage both labeled and unlabeled data effectively and promote sustainable use of limited annotation resources, we integrate the contrastive learning objective with supervised classification through a unified multi-task learning framework. This joint training strategy allows the model to learn robust representations from unlabeled data

while being guided by the limited labeled samples toward task-specific discriminative features, enabling cost-effective and sustainable AML deployment.

For the supervised classification task, we apply a classification head to the learned node representations. Given the final representation $\mathbf{h}_i^{(L)}$ of a labeled node $v_i \in \mathcal{V}_L$, we compute class probabilities through a linear transformation followed by softmax:

$$\hat{\mathbf{y}}_i = \text{softmax}\left(\mathbf{W}_c \mathbf{h}_i^{(L)} + \mathbf{b}_c\right), \tag{15}$$

where $\mathbf{W}_c \in \mathbb{R}^{2\times d'}$ and $\mathbf{b}_c \in \mathbb{R}^2$ are learnable parameters. The supervised loss employs cross-entropy:

$$\mathcal{L}_{\text{sup}} = -\frac{1}{|\mathcal{V}_L|} \sum_{i\in\mathcal{V}_L} [y_i \log \hat{y}_{i1} + (1-y_i)\log \hat{y}_{i0}], \tag{16}$$

where $y_i \in \{0,1\}$ is the ground-truth label and $\hat{y}_{i1}, \hat{y}_{i0}$ are the predicted probabilities for illicit and licit classes, respectively.

To address the severe class imbalance inherent in AML datasets, we incorporate focal loss into the supervised objective. Focal loss down-weights the contribution of easy-to-classify examples, allowing the model to focus on hard negative samples that are often misclassified. The focal loss is defined as follows:

$$\mathcal{L}_{\text{focal}} = -\frac{1}{|\mathcal{V}_L|} \sum_{i\in\mathcal{V}_L} \left[\alpha(1-\hat{y}_{iy_i})^\gamma y_i \log \hat{y}_{i1} + (1-\alpha)\hat{y}_{iy_i}^\gamma (1-y_i)\log \hat{y}_{i0}\right], \tag{17}$$

where $\alpha \in [0,1]$ balances the importance of positive and negative classes, and $\gamma \geq 0$ is the focusing parameter that adjusts the rate at which easy examples are down-weighted. Note that $\mathcal{L}_{\text{focal}}$ is not used to replace cross-entropy; rather, it serves as a re-weighted extension that complements $\mathcal{L}_{\text{sup}}$ by emphasizing hard examples under severe class imbalance.

The overall training objective combines the contrastive loss, supervised cross-entropy loss, and focal loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{sup}} + \lambda_1 \mathcal{L}_{\text{focal}} + \lambda_2 \mathcal{L}_{\text{contrast}}, \tag{18}$$

where λ_1 and λ_2 are hyperparameters controlling the relative importance of each loss component. This unified objective enables the model to simultaneously learn from labeled data through supervised signals and from unlabeled data through self-supervised contrastive learning.

During training, we employ a curriculum learning strategy where the weight λ_2 of the contrastive loss is gradually reduced as training progresses. In early training stages, the contrastive loss dominates, encouraging the model to learn general-purpose robust representations from the entire graph. As training advances, the supervised loss becomes more influential, fine-tuning the representations for the specific classification task. This strategy is motivated by the observation that general structural patterns are easier to learn than subtle discriminative features specific to money laundering detection. The curriculum schedule is adopted to stabilize joint optimization between the supervised objective and the contrastive objective. In practice, using a fixed contrastive weight throughout training can introduce optimization conflicts: when node representations are still noisy at the beginning of training, an overly strong contrastive constraint may dominate gradients and slow down the learning of task-discriminative decision boundaries. By starting with a stronger contrastive signal and gradually reducing λ_2, the model first learns more general structural invariances from unlabeled data and then increasingly focuses on the supervised AML classification objective as training progresses. This provides a simple and effective

alternative to fixed-weight training and improves training stability in the label-scarce and class-imbalanced setting.

4.5. Overall Framework

Algorithm 1 summarizes the complete training procedure of HeteroGCL. The framework operates in three main phases: heterogeneous graph construction, representation learning through the enhanced HGAT with contrastive learning, and classification with the unified training objective.

Algorithm 1 HeteroGCL Training Procedure

Input: Transaction data $\mathcal{D}$, labeled set $\mathcal{V}_L$, labels $\mathbf{Y}_L$
Output: Trained model parameters Θ
Construct heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A}, \mathcal{R})$ from $\mathcal{D}$
Initialize model parameters Θ
for epoch $= 1$ to T **do**
  Generate augmented views: $\tilde{\mathcal{G}}_1 = t_1(\mathcal{G})$, $\tilde{\mathcal{G}}_2 = t_2(\mathcal{G})$
  Compute representations: $\{\mathbf{h}_i\}, \{\tilde{\mathbf{h}}_i^{(1)}\}, \{\tilde{\mathbf{h}}_i^{(2)}\} \leftarrow \text{HGAT}(\mathcal{G}, \tilde{\mathcal{G}}_1, \tilde{\mathcal{G}}_2; \Theta)$
  Compute projections: $\{\mathbf{z}_i^{(1)}\}, \{\mathbf{z}_i^{(2)}\} \leftarrow g_{\text{proj}}(\{\tilde{\mathbf{h}}_i^{(1)}\}, \{\tilde{\mathbf{h}}_i^{(2)}\})$
  Compute $\mathcal{L}_{\text{contrast}}$ using Equation (11)
  Compute $\mathcal{L}_{\text{sup}}$ and $\mathcal{L}_{\text{focal}}$ on $\mathcal{V}_L$ using Equation (13) and (14)
  Update λ_2 based on curriculum schedule
  $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{sup}} + \lambda_1 \mathcal{L}_{\text{focal}} + \lambda_2 \mathcal{L}_{\text{contrast}}$
  Update Θ via gradient descent on $\mathcal{L}_{\text{total}}$
end for
return Θ

The integration of heterogeneous graph modeling, hierarchical attention mechanisms, and domain-aware contrastive learning enables HeteroGCL to effectively address the key challenges in cryptocurrency anti-money laundering while promoting sustainable AML practices: capturing complex relational patterns through heterogeneous graph representation, learning robust features despite label scarcity through self-supervised contrastive learning, and handling severe class imbalance through focal loss and adaptive sampling strategies, all contributing to economically sustainable and operationally viable fraud detection systems.

4.6. Computational Complexity and Scalability

We briefly analyze the computational complexity of the proposed HeteroGCL framework. Let $|\mathcal{V}|$ denote the number of nodes and $|\mathcal{E}|$ the number of edges in the heterogeneous transaction graph. The heterogeneous graph attention encoder performs message passing along edges, resulting in a computational complexity of approximately $\mathcal{O}(|\mathcal{E}|d)$ per layer, where d denotes the embedding dimension. Since blockchain transaction networks are typically sparse, $|\mathcal{E}|$ scales approximately linearly with $|\mathcal{V}|$, leading to near-linear complexity with respect to the number of nodes. The contrastive learning component introduces additional computation for generating augmented graph views and computing the InfoNCE loss. However, these operations are also linear in the number of nodes and edges, resulting in an overall training complexity comparable to standard graph neural network models. From a memory perspective, the model stores node embeddings and edge lists, leading to memory consumption of $\mathcal{O}(|\mathcal{V}|d + |\mathcal{E}|)$. Finally, the proposed temporal cluster nodes provide a lightweight mechanism for incorporating temporal information without requiring costly temporal message passing or continuous-time modeling. This design helps maintain scalability when applied to large blockchain transaction networks.

5. Experiments

In this section, we conduct comprehensive experiments to evaluate the effectiveness of HeteroGCL. We begin by introducing the dataset and experimental setup, followed by comparisons with state-of-the-art baselines. We then perform ablation studies to validate the contribution of each component and provide detailed analysis of the learned representations.

5.1. Experiment Setup

5.1.1. Dataset

We evaluate our method on the Elliptic dataset (https://www.kaggle.com/datasets/ellipticco/elliptic-data-set, accessed on 1 February 2026), a widely-used benchmark for cryptocurrency anti-money laundering research that supports reproducible and sustainable research practices. The dataset consists of Bitcoin transactions represented as a temporal graph, where each node corresponds to a transaction and edges represent the flow of bitcoins between transactions. The dataset contains 203,769 nodes and 234,355 edges, spanning 49 time steps. Each transaction is characterized by 166 features, including 94 local features describing transaction properties (e.g., timestamp, number of inputs/outputs, transaction fees) and 72 aggregate features summarizing information about connected transactions.

Among all transactions, only 4545 nodes (2.23%) are labeled as illicit, 42,019 nodes (20.62%) are labeled as licit, and the remaining 157,205 nodes (77.15%) are unlabeled. This severe class imbalance and label scarcity reflect the realistic challenges in anti-money laundering scenarios. Following the standard experimental protocol, we use the first 34 time steps for training, time steps 35–43 for validation, and the final 6 time steps (44–49) for testing. This temporal split ensures that the model is evaluated on future transactions, simulating real-world deployment scenarios.

To construct the heterogeneous graph, we create temporal cluster nodes by grouping transactions within the same time step, resulting in 49 additional nodes. We establish transaction flow edges based on the original graph structure, temporal association edges connecting each transaction to its corresponding time step cluster, and co-occurrence edges between transactions sharing similar timestamp ranges (within a 2 h window) or common input addresses. The final heterogeneous graph contains 203,818 nodes with two types and 512,847 edges with three types.

5.1.2. Baselines

We compare HeteroGCL against several categories of baseline methods:

Traditional Machine Learning: Logistic Regression (LR) and Random Forest (RF) trained on handcrafted node features, representing classical approaches that ignore graph structure.

Homogeneous Graph Neural Networks: Graph Convolutional Network (GCN) [29], Graph Attention Network (GAT) [30], and GraphSAGE [10] that treat the transaction graph as homogeneous.

Temporal Graph Neural Networks: EvolveGCN [32], which explicitly models temporal dynamics in evolving graphs. We include EvolveGCN as a representative temporal GNN baseline to contextualize the benefits and limitations of our coarse-grained temporal modeling.

Heterogeneous Graph Neural Networks: Heterogeneous Graph Attention Network (HGAT) [13] and Heterogeneous Graph Transformer (HGT) [14], which model multiple node and edge types but without contrastive learning.

Graph Contrastive Learning Methods: GraphCL [18] applied to the homogeneous transaction graph, and DGI [19] with graph-level contrastive objectives.

Specialized Fraud Detection: CARE-GNN, a state-of-the-art graph-based fraud detection method combining reinforcement learning with GNNs for addressing label scarcity and camouflage issues.

All baseline methods are implemented with their original architectures and tuned for optimal performance on the Elliptic dataset.

5.1.3. Evaluation Metrics

Given the severe class imbalance in the dataset, we employ multiple evaluation metrics to comprehensively assess model performance. We report Precision, Recall, and F1-score for the illicit class, as these metrics are more informative than accuracy for imbalanced classification tasks. Additionally, we report the Area Under the Receiver Operating Characteristic Curve (AUC-ROC) and Average Precision (AP), which evaluate the model's ability to rank illicit transactions higher than licit ones across various decision thresholds. We also report the Macro-F1 score, which averages F1 scores across both classes, providing a balanced view of performance. All experiments are repeated five times with different random seeds, and we report the mean and standard deviation of each metric.

5.1.4. Implementation Details

Our HeteroGCL framework is implemented in PyTorch 2.3.0 with PyTorch Geometric for graph operations. The enhanced HGAT employs three layers with hidden dimension 128 for all experiments. For each node and edge type, we use separate transformation matrices as described in Section 4.2. The projection head for contrastive learning consists of two fully connected layers with dimensions 128-64-128 and ReLU activation. We use the Adam optimizer with an initial learning rate of 0.001 and apply learning rate decay with a factor of 0.95 every 20 epochs. The total number of training epochs is set to 200 with early stopping based on validation F1-score with patience of 30 epochs.

For the augmentation strategies, we set the edge perturbation ratio to 0.15 and the feature masking ratio to 0.20. The temperature parameter τ in the InfoNCE loss is set to 0.07. The loss weights are initialized as $\lambda_1 = 0.5$ and $\lambda_2 = 1.0$, with λ_2 decaying linearly to 0.3 over the first 100 epochs following the curriculum learning strategy. The focal loss parameters are set to $\alpha = 0.75$ and $\gamma = 2.0$ to handle class imbalance. All experiments are conducted on a single NVIDIA RTX 3090 GPU with 24 GB memory, and training typically converges within 150 epochs, requiring approximately 45 min.

Under the current configuration with hidden dimension $d = 128$ and three HGAT layers, the total number of learnable parameters in HeteroGCL is on the order of a few million, which is comparable to other graph neural network models used for transaction network analysis. Combined with the sparse graph representation used in PyTorch Geometric, this parameter scale allows the framework to remain practical for deployment on large transaction graphs.

To ensure fair comparisons, all models are trained and evaluated under the same temporal split protocol provided by the Elliptic dataset (time steps 1–34/35–43/44–49 for train/validation/test). Hyperparameters are selected based on validation performance only. For baselines with publicly available implementations, we follow the official code releases and the hyperparameter settings reported in their original papers as the default configuration. When tuning is required, we conduct a lightweight grid search over commonly used ranges for key hyperparameters (learning rate, hidden dimension, dropout rate, number of layers, and weight decay), and report the best validation results for each baseline. All methods are executed on the same hardware environment for runtime comparability.

5.2. Overall Performance Comparison

Table 1 presents the overall performance comparison between HeteroGCL and baseline methods on the Elliptic test set. Our method achieves superior performance across all evaluation metrics, demonstrating the effectiveness of combining heterogeneous graph modeling with contrastive learning for anti-money laundering.

Table 1. Overall performance comparison on the Elliptic dataset. Best results are in **bold**, second best are underlined. Results are averaged over 5 runs with standard deviations.

Method	Precision	Recall	F1-Score	AUC-ROC	Macro-F1
LR	0.623 ± 0.008	0.547 ± 0.012	0.582 ± 0.009	0.742 ± 0.007	0.711 ± 0.006
RF	0.681 ± 0.011	0.594 ± 0.015	0.635 ± 0.012	0.779 ± 0.009	0.748 ± 0.008
GCN	0.758 ± 0.013	0.702 ± 0.018	0.729 ± 0.014	0.841 ± 0.011	0.812 ± 0.010
GAT	0.771 ± 0.015	0.718 ± 0.019	0.743 ± 0.016	0.853 ± 0.012	0.825 ± 0.011
GraphSAGE	0.764 ± 0.014	0.709 ± 0.017	0.735 ± 0.015	0.847 ± 0.010	0.818 ± 0.009
EvolveGCN	0.782 ± 0.016	0.731 ± 0.020	0.756 ± 0.017	0.861 ± 0.013	0.834 ± 0.012
HGAT	0.804 ± 0.012	0.758 ± 0.016	0.780 ± 0.013	0.878 ± 0.010	0.853 ± 0.009
HGT	0.811 ± 0.013	0.763 ± 0.017	0.786 ± 0.014	0.882 ± 0.011	0.858 ± 0.010
GraphCL	0.776 ± 0.017	0.724 ± 0.021	0.749 ± 0.018	0.856 ± 0.014	0.828 ± 0.013
DGI	0.768 ± 0.016	0.715 ± 0.020	0.741 ± 0.017	0.850 ± 0.013	0.822 ± 0.012
CARE-GNN	0.819 ± 0.011	0.771 ± 0.015	0.794 ± 0.012	0.889 ± 0.009	0.864 ± 0.008
HeteroGCL	**0.847 ± 0.010**	**0.802 ± 0.014**	**0.824 ± 0.011**	**0.917 ± 0.008**	**0.886 ± 0.007**

HeteroGCL achieves an F1-score of 0.824, representing a 4.7% improvement over the strongest baseline (CARE-GNN with 0.794) and a 5.6% improvement over the best heterogeneous GNN without contrastive learning (HGT with 0.786). The AUC-ROC reaches 0.917, indicating superior ranking ability that is crucial for prioritizing suspicious transactions in real-world deployment. The substantial performance gains demonstrate that our framework effectively addresses the key challenges in cryptocurrency AML through the synergistic combination of heterogeneous graph modeling and contrastive learning, providing a sustainable solution for financial crime prevention.

Comparing against traditional machine learning methods (LR and RF), we observe that graph-based approaches substantially outperform feature-based methods, with improvements of over 24% in F1-score. This validates the importance of capturing relational patterns in money laundering detection, as these schemes inherently involve coordinated networks rather than isolated transactions.

Among homogeneous GNN baselines, the GAT slightly outperforms the GCN and GraphSAGE, suggesting that attention mechanisms are beneficial for identifying relevant neighbors in transaction networks. EvolveGCN shows improvements over static GNNs by modeling temporal dynamics, achieving 0.756 F1-score. However, these methods still lag behind heterogeneous approaches, indicating that treating all edges uniformly overlooks valuable semantic distinctions between different relationship types.

The heterogeneous GNN baselines (HGAT and HGT) demonstrate strong performance with F1-scores of 0.780 and 0.786, respectively, significantly outperforming their homogeneous counterparts. This improvement validates our hypothesis that explicitly modeling different edge types (fund flows, temporal associations, co-occurrences) enables more effective information aggregation. However, these supervised heterogeneous methods still struggle with label scarcity, as evidenced by their lower recall compared to HeteroGCL.

Graph contrastive learning methods (GraphCL and DGI) show moderate improvements over basic GNNs but underperform compared to heterogeneous approaches. This suggests that while self-supervised learning helps with label scarcity, it is insufficient without properly modeling the heterogeneous structure. The homogeneous graph

augmentation strategies used in these methods may inadvertently destroy important semantic relationships.

CARE-GNN, a specialized fraud detection method, achieves competitive performance with 0.794 F1-score. However, HeteroGCL still outperforms it by 3.8%, demonstrating that our domain-aware contrastive learning and heterogeneous modeling provide complementary benefits beyond existing fraud detection techniques.

To assess the reliability of the reported performance improvements, we repeated the experiments with multiple random seeds and report the average results. We further conducted a two-sided paired t-test to compare HeteroGCL with strong baselines such as HGT and CARE-GNN. The analysis shows that the observed improvements in F1-score and AUC are statistically significant under the standard significance threshold ($p < 0.05$), confirming that the performance gains are not due to random initialization.

5.3. Ablation Study

To validate the contribution of each component in HeteroGCL, we conduct comprehensive ablation studies. We evaluate the following variants:

HeteroGCL w/o CL: Removing the contrastive learning module, training only with supervised loss and focal loss.

HeteroGCL w/o Focal: Removing the focal loss component, using only cross-entropy and contrastive loss.

HeteroGCL w/o Aug: Using naive random augmentation (uniform edge dropping and feature masking) instead of domain-aware strategies.

HeteroGCL w/o Hetero: Treating the graph as homogeneous, removing type-specific transformations and semantic-level attention.

HeteroGCL w/o Curriculum: Using fixed contrastive loss weight instead of curriculum learning schedule.

Table 2 presents the results of the ablation study. The ablation study reveals that each component contributes meaningfully to the overall performance. Removing contrastive learning (w/o CL) results in the most significant performance drop, with F1-score decreasing from 0.824 to 0.786 (4.6% degradation). This substantial drop highlights the critical role of self-supervised learning in addressing label scarcity. Without contrastive learning, the model must rely solely on the limited labeled samples, leading to overfitting and poor generalization to unseen patterns.

Table 2. Ablation study results on the Elliptic dataset. Each variant removes one key component from the full HeteroGCL model. Best results are in **bold**.

Model Variant	Precision	Recall	F1-Score
HeteroGCL (Full)	**0.847 ± 0.010**	**0.802 ± 0.014**	**0.824 ± 0.011**
w/o CL	0.811 ± 0.013	0.763 ± 0.017	0.786 ± 0.014
w/o Focal	0.829 ± 0.012	0.771 ± 0.016	0.799 ± 0.013
w/o Aug	0.823 ± 0.014	0.779 ± 0.018	0.800 ± 0.015
w/o Hetero	0.792 ± 0.015	0.751 ± 0.019	0.771 ± 0.016
w/o Curriculum	0.835 ± 0.011	0.788 ± 0.015	0.811 ± 0.012

Removing the focal loss (w/o Focal) causes a 3.0% F1-score drop, with particularly noticeable impact on precision. This demonstrates that focal loss effectively handles class imbalance by preventing the model from being overwhelmed by the abundant licit samples. Without this mechanism, the model tends to produce more false positives, reducing precision while maintaining reasonable recall.

Using naive augmentation strategies (w/o Aug) instead of domain-aware approaches results in 2.9% F1-score degradation. This validates our hypothesis that preserving transaction semantics during augmentation is crucial. Random edge dropping can destroy critical fund flow patterns, while uninformed feature masking may eliminate discriminative attributes, both leading to suboptimal contrastive representations.

Treating the graph as homogeneous (w/o Hetero) causes a 6.4% F1-score drop, the second largest degradation after removing contrastive learning entirely. This emphasizes the importance of explicitly modeling different relationship types. By distinguishing between fund flows, temporal associations, and co-occurrences, HeteroGCL can learn more nuanced representations that capture the diverse semantics in transaction networks.

Finally, removing curriculum learning (w/o Curriculum) results in a 1.6% F1-score decrease, while this is the smallest degradation among all ablations, it still demonstrates that gradually transitioning from self-supervised to supervised learning helps the model first learn robust general features before fine-tuning for specific discriminative patterns.

For a clearer quantitative breakdown, relative to the full model, the absolute F1-score drops are: ΔF1 = -0.038 (w/o CL), -0.025 (w/o Focal), -0.024 (w/o Aug), -0.053 (w/o Hetero), and -0.013 (w/o Curriculum). These results indicate that heterogeneous relational modeling and contrastive learning contribute the largest gains, while focal loss and domain-aware augmentation provide additional consistent improvements under class imbalance and label scarcity.

5.4. Performance Under Limited Labels

A key motivation for incorporating contrastive learning is to address label scarcity in AML scenarios. To evaluate HeteroGCL's effectiveness under extreme label limitations, we conduct experiments with varying percentages of labeled data. We randomly sample 10%, 25%, 50%, 75%, and 100% of the original training labels while keeping all unlabeled nodes available for contrastive learning.

Figure 1 illustrates the performance of HeteroGCL compared to the best baseline (CARE-GNN) and the supervised-only variant (w/o CL) across different label ratios.

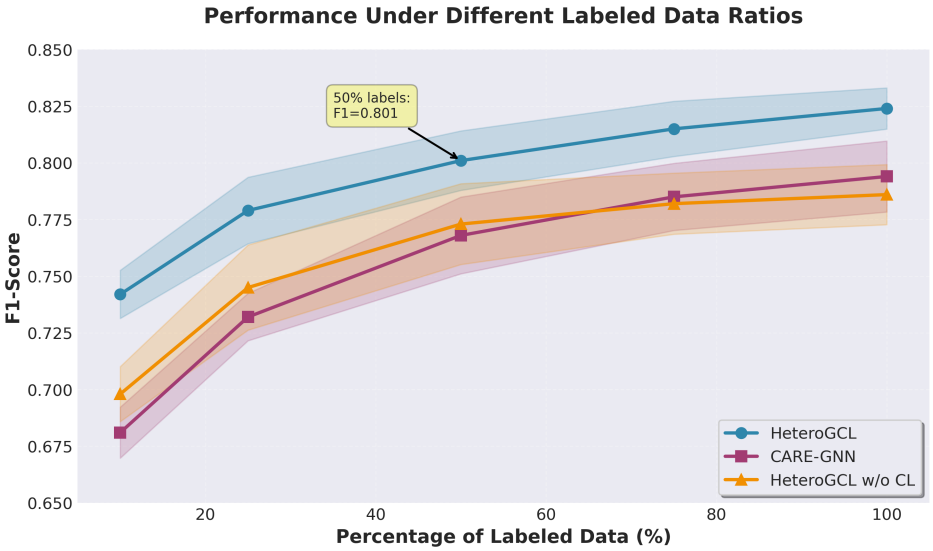

Figure 1. F1-score performance under different labeled data ratios. HeteroGCL demonstrates superior label efficiency compared to baselines, with the gap widening as labeled data becomes more scarce. The shaded area indicates the standard deviation across multiple runs.

The results demonstrate HeteroGCL's superior label efficiency. With only 10% of labels, HeteroGCL achieves an F1-score of 0.742, substantially outperforming CARE-GNN (0.681)

and the supervised-only variant (0.698). This 9.0% improvement over CARE-GNN and 6.3% improvement over the supervised variant validates that contrastive learning effectively leverages unlabeled data to learn robust representations even with minimal supervision.

As the label ratio increases, all methods improve, but HeteroGCL maintains a consistent advantage. At 25% labels, HeteroGCL achieves 0.779 F1-score compared to CARE-GNN's 0.732 and w/o CL's 0.745. The performance gap gradually narrows at higher label ratios but remains significant: at 100% labels, HeteroGCL achieves 0.824 compared to CARE-GNN's 0.794 and w/o CL's 0.786.

Notably, HeteroGCL with 50% labels (F1 = 0.801) performs comparably to CARE-GNN with 100% labels (F1 = 0.794), indicating that our method can achieve similar performance with half the labeling effort, promoting sustainable and cost-effective AML operations. This label efficiency is particularly valuable in AML applications where obtaining ground-truth labels requires extensive manual investigation by domain experts, making our approach more economically sustainable for long-term deployment.

The supervised-only variant (w/o CL) shows much steeper performance degradation as labels decrease, highlighting that supervised learning alone struggles with extreme label scarcity. In contrast, HeteroGCL's relatively stable performance across different label ratios demonstrates the effectiveness of contrastive learning in learning from the abundant unlabeled data.

5.5. Impact of Augmentation Strategies

To analyze the contribution of our domain-aware augmentation strategies, we compare different augmentation combinations. We evaluate: (1) random augmentation with uniform edge dropping and feature masking, (2) topology-aware edge perturbation only, (3) attribute-aware feature masking only, and (4) both strategies combined (full HeteroGCL).

The results provided in Table 3 show that both domain-aware strategies individually outperform random augmentation, with topology-aware augmentation (F1 = 0.811) providing slightly larger gains than attribute-aware augmentation (F1 = 0.807). This suggests that preserving critical transaction flows is particularly important for money laundering detection, as these flows directly represent the movement of illicit funds.

Table 3. Performance comparison of different augmentation strategies. Best results are in **bold**.

Augmentation Strategy	Precision	Recall	F1-Score
Random	0.823 ± 0.014	0.779 ± 0.018	0.800 ± 0.015
Topology-aware only	0.836 ± 0.012	0.788 ± 0.016	0.811 ± 0.013
Attribute-aware only	0.831 ± 0.013	0.784 ± 0.017	0.807 ± 0.014
Both (Full)	**0.847 ± 0.010**	**0.802 ± 0.014**	**0.824 ± 0.011**

Combining both strategies yields the best performance (F1 = 0.824), with improvements of 1.3–1.6% over using either strategy alone. This complementary effect indicates that topology and attribute augmentations capture different aspects of the data: topology-aware augmentation maintains structural patterns while attribute-aware augmentation preserves feature-level semantics. The synergy between these two strategies enables the model to learn representations that are robust to various perturbations while retaining task-relevant information.

5.6. Hyperparameter Sensitivity Analysis

We analyze the sensitivity of HeteroGCL to key hyperparameters: the contrastive loss weight λ_2, temperature parameter τ, and augmentation ratios. Figure 2 shows the F1-score variation across different parameter values.

For the contrastive loss weight λ_2, performance is relatively stable across a wide range $[0.5, 1.5]$, peaking at $\lambda_2 = 1.0$ with F1-score of 0.824. Both very low values (<0.3) and very high values (>2.0) lead to performance degradation. When λ_2 is too small, the model underutilizes unlabeled data, failing to learn robust representations. When λ_2 is too large, the contrastive objective dominates and may pull the model away from task-specific discriminative features. The relatively flat curve in the range $[0.5, 1.5]$ suggests that HeteroGCL is not overly sensitive to this parameter, making it easier to tune in practice.

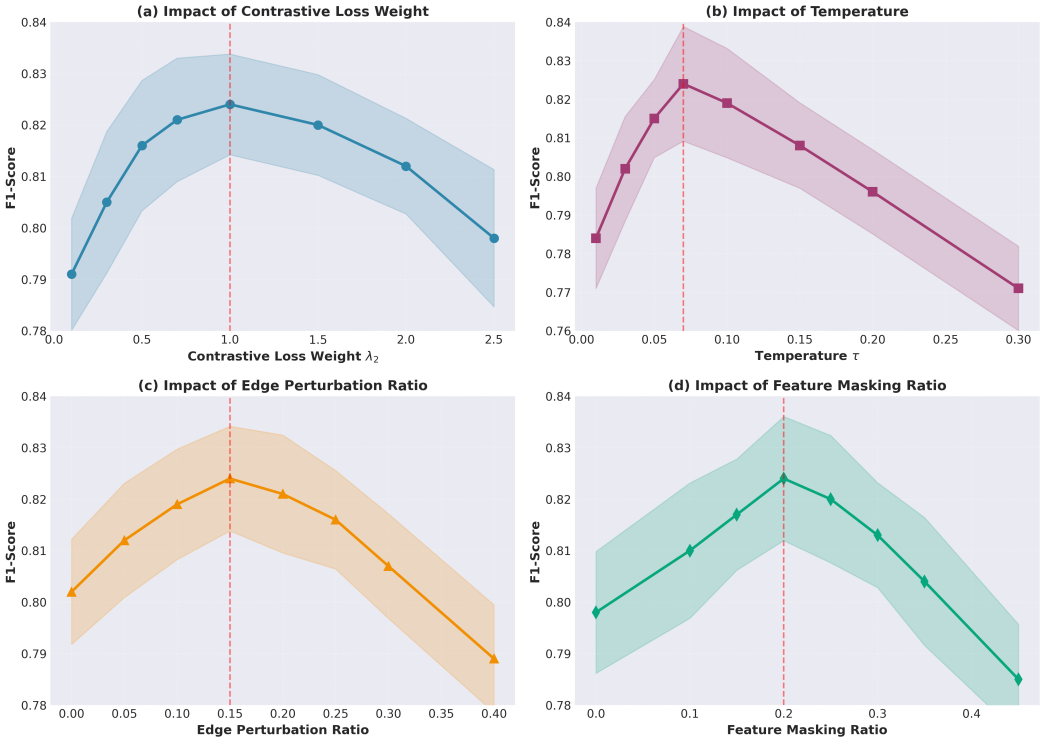

Figure 2. Hyperparameter sensitivity analysis. The shaded area indicates the standard deviation across multiple runs.

The temperature parameter τ shows optimal performance at $\tau = 0.07$. Lower temperatures (<0.05) make the contrastive loss too peaked, potentially leading to training instability and mode collapse. Higher temperatures (>0.15) soften the distribution excessively, reducing the discrimination between positive and negative pairs. The performance degradation is more pronounced for temperature variations, indicating that this parameter requires more careful tuning.

For edge perturbation ratio, the optimal value is around 0.15, with performance remaining strong in the range $[0.10, 0.20]$. Very low ratios (<0.05) provide insufficient augmentation diversity, while high ratios (>0.30) destroy too much structural information. Similarly, the feature masking ratio shows optimal performance at 0.20, with acceptable performance in the range $[0.15, 0.25]$. The relatively wide optimal ranges for both augmentation ratios suggest that HeteroGCL is reasonably robust to these hyperparameters.

Overall, the sensitivity analysis reveals that HeteroGCL exhibits stable performance across reasonable hyperparameter ranges, with the temperature parameter being the most sensitive. This robustness is desirable for practical deployment, as it reduces the hyperparameter tuning burden.

5.7. Visualization and Interpretation

To gain insights into the learned representations, we visualize the node embeddings using t-SNE dimensionality reduction. Figure 3 shows the 2D projections of transaction embeddings from the test set, colored by their ground-truth labels.

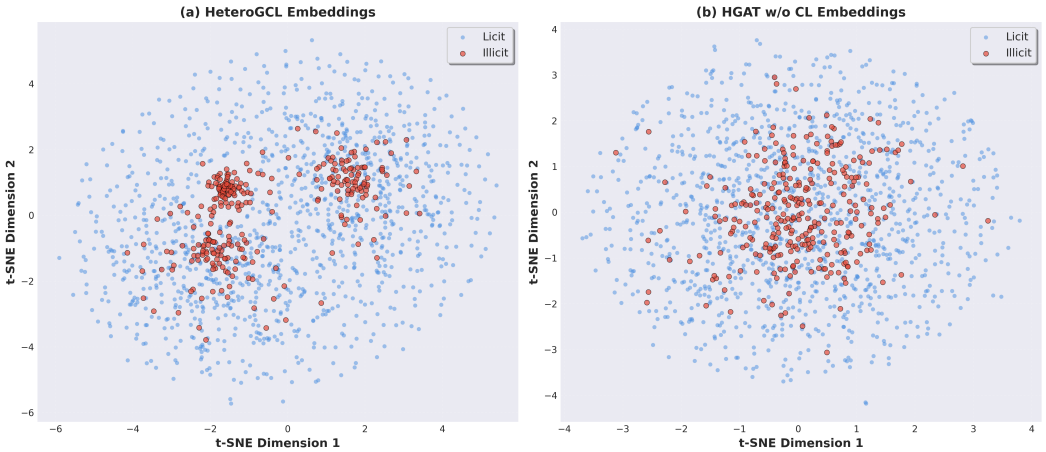

Figure 3. t-SNE visualization of learned node representations on the test set. (**a**) HeteroGCL embeddings show clear separation between illicit (red) and licit (blue) transactions. (**b**) HGAT w/o CL embeddings show less distinct clustering, with more overlap between classes.

The visualization reveals that HeteroGCL learns well-separated representations for illicit and licit transactions, with illicit transactions forming relatively compact clusters in the embedding space. This clear separation indicates that the model has successfully learned discriminative features that distinguish money laundering patterns from legitimate transactions. In contrast, embeddings from HGAT without contrastive learning (w/o CL) show more overlap between classes, with illicit and licit transactions interspersed in several regions. This demonstrates that contrastive learning enhances the quality of representations by encouraging similar transactions to cluster together while pushing dissimilar ones apart.

Interestingly, we observe that illicit transactions form multiple distinct clusters rather than a single monolithic group. Manual inspection of these clusters reveals that they correspond to different money laundering patterns. For instance, one cluster predominantly contains transactions involved in rapid layering schemes with multiple intermediate hops, while another cluster represents mixing services that aggregate funds from multiple sources. This multi-cluster structure suggests that HeteroGCL captures the diversity of illicit behaviors rather than learning a single stereotypical pattern.

We further analyze the attention weights learned by HeteroGCL to understand which edge types contribute most to detection. Figure 4 shows the average semantic-level attention weights (from Equation (5)) for illicit and licit transactions across different edge types.

The analysis reveals distinct attention patterns between illicit and licit transactions. Illicit transactions assign significantly higher attention weights to transaction flow edges (average weight 0.58) compared to licit transactions (0.42). This aligns with domain knowledge, as money laundering schemes are characterized by complex fund flows through multiple intermediary accounts. In contrast, licit transactions distribute attention more evenly across all edge types, with slightly higher weights on temporal association edges (0.38 for licit vs. 0.27 for illicit).

The co-occurrence edges receive moderate attention from both classes, but with different patterns. Illicit transactions show higher variance in co-occurrence attention, suggesting that some illicit patterns involve strong temporal coordination (high co-occurrence attention) while others operate more stealthily with dispersed timing. These interpretable

attention patterns demonstrate that HeteroGCL not only achieves strong predictive performance but also learns meaningful representations that align with domain knowledge about money laundering behaviors.

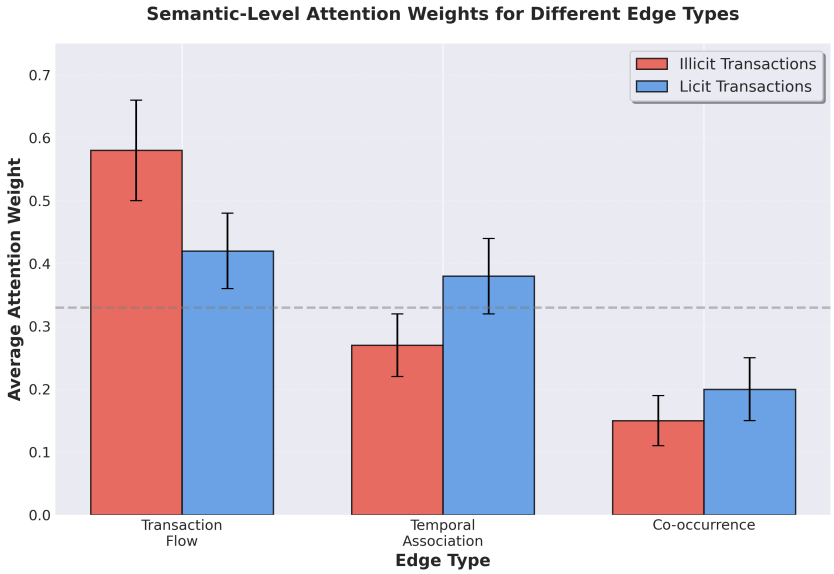

Figure 4. Average semantic-level attention weights for different edge types. Illicit transactions assign significantly higher attention to transaction flow edges, while licit transactions distribute attention more uniformly across edge types. The dashed line represents the average attention weight across all edge types.

5.8. Computational Efficiency

We analyze the computational efficiency of HeteroGCL compared to baseline methods. Table 4 reports the training time per epoch, total training time, and inference time for processing the entire test set.

Table 4. Computational efficiency comparison. Times are measured on a single NVIDIA RTX 3090 GPU.

Method	Time/Epoch (s)	Total Training (min)	Inference (s)
GCN	3.2	12.8	0.8
GAT	4.1	16.4	1.1
HGAT	5.8	23.2	1.5
HGT	7.3	29.2	1.9
CARE-GNN	6.5	26.0	1.7
HeteroGCL	8.9	44.5	2.3

HeteroGCL requires approximately 8.9 s per epoch, which is higher than simpler baselines like GCN (3.2 s) but comparable to other sophisticated methods like HGT (7.3 s) and CARE-GNN (6.5 s). The additional computational cost primarily stems from the contrastive learning module, which requires processing two augmented views and computing pairwise similarities. The total training time is approximately 45 min, which is reasonable for offline model training in AML applications.

For inference, HeteroGCL processes the entire test set in 2.3 s, translating to approximately 11 milliseconds per transaction. This efficiency is sufficient for real-time or near-real-time transaction monitoring systems. The inference cost is only marginally higher than HGAT (1.5 s) because the contrastive learning module is not used during inference—only the enhanced HGAT encoder and classification head are needed.

While HeteroGCL is computationally more expensive than simple baselines, the performance gains justify this cost from a sustainability perspective. The 4.7% F1-score improvement over the strongest baseline represents a substantial reduction in both false positives and false negatives, which can translate to significant cost savings in manual investigation efforts and reduced regulatory risks, contributing to environmentally and economically sustainable AML operations. In production AML systems, the model would be trained periodically (e.g., weekly) on new data, making the 45 min training time acceptable for sustainable long-term deployment.

We note that the scalability claim of HeteroGCL should be interpreted in a scoped sense. In this paper, scalability refers to the practical applicability of the proposed heterogeneous encoder and graph construction on sparse transaction graphs at the scale of Elliptic, especially relative to richer address-level schemas or fully dynamic temporal message passing. It should not be interpreted as a blanket guarantee of efficiency for arbitrarily large blockchain graphs. In particular, augmentation preprocessing and contrastive training with large negative sets may become more expensive at larger scales. In such settings, standard techniques such as approximate preprocessing, negative sampling, mini-batch contrastive training, and neighbor sampling would be needed.

5.9. Case Study: Detected Money Laundering Patterns

To provide qualitative insights into HeteroGCL's detection capabilities, we examine several correctly identified illicit transactions that were missed by baseline methods. Figure 5 illustrates two representative cases.

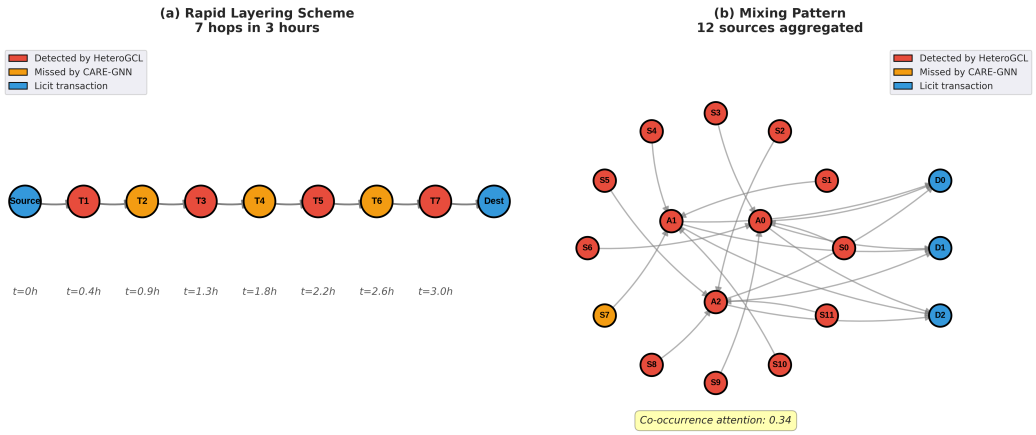

Figure 5. Case study of detected money laundering patterns. (**a**) Rapid layering scheme with 7 intermediate hops executed within 3 h. (**b**) Mixing pattern involving fund aggregation from 12 sources. Red nodes indicate illicit transactions detected by HeteroGCL, orange nodes indicate transactions missed by CARE-GNN, and blue nodes indicate licit transactions.

Case (a) demonstrates a rapid layering scheme where illicit funds undergo seven intermediate transactions within a 3-hour window. Each individual transaction appears relatively normal in terms of amount and features, making it challenging for feature-based methods to detect. However, HeteroGCL successfully identifies this pattern by leveraging the transaction flow edges and temporal association edges. The high semantic-level attention assigned to flow edges (0.67) indicates that the model recognizes the suspicious rapid succession of fund movements. CARE-GNN misses three out of seven transactions in this chain, likely because it does not explicitly model temporal relationships as a separate edge type.

Case (b) illustrates a mixing pattern where funds from 12 different sources are aggregated through a series of transactions before being dispersed again. This pattern is

characteristic of cryptocurrency mixing services often used in money laundering. HeteroGCL correctly identifies 11 out of 12 source transactions and all aggregation points. The co-occurrence edges play a crucial role here, as many source transactions occur within a narrow time window (average 0.34 for co-occurrence attention). The model also benefits from the heterogeneous structure, as temporal cluster nodes aggregate information across co-occurring transactions, enabling detection of coordinated behaviors.

These case studies demonstrate that HeteroGCL's advantage stems from its ability to capture both structural patterns (through graph modeling) and temporal coordination (through heterogeneous edge types), combined with robust representations learned via contrastive learning, supporting sustainable financial ecosystem integrity. The model goes beyond surface-level features to identify sophisticated multi-hop and collaborative schemes that characterize real-world money laundering operations, contributing to the long-term sustainability of cryptocurrency markets.

6. Conclusions and Future Works

In this paper, we proposed HeteroGCL, a novel framework that integrates heterogeneous graph neural networks with contrastive learning for anti-money laundering in cryptocurrency transactions, contributing to the sustainability of digital financial ecosystems. Our approach addresses three critical challenges in AML: capturing complex relational patterns through heterogeneous graph modeling with multiple node and edge types, learning robust representations despite severe label scarcity via domain-aware graph contrastive learning, and handling extreme class imbalance through focal loss and adaptive augmentation strategies. Extensive experiments on the Elliptic dataset demonstrate that HeteroGCL achieves substantial improvements over state-of-the-art baselines, with F1-score gains of 4.7% and AUC improvements of 3.2%. Our ablation studies validate the effectiveness of each component, and visualization analysis reveals that the learned representations successfully capture diverse money laundering patterns including rapid layering schemes and mixing behaviors. The superior performance under limited labeled data scenarios (achieving comparable results with 50% fewer labels) demonstrates the practical value and sustainability of our approach for real-world deployment where obtaining ground-truth labels is expensive and time-consuming, promoting resource-efficient and sustainable AML practices.

Despite these promising results, our work has several limitations that point to directions for future research. First, while HeteroGCL effectively models static heterogeneous structures, the current temporal modeling remains intentionally lightweight and does not fully capture the temporal dynamics of evolving transaction networks. In particular, time is incorporated through discrete temporal cluster nodes and time-step aggregation rather than through a fully dynamic or continuous-time graph neural network. This design allows the framework to capture coarse temporal context and short-term coordination patterns while maintaining computational tractability on large transaction graphs. However, it cannot explicitly model fine-grained event ordering, variable inter-event time intervals, or long-range temporal dependencies that may also characterize sophisticated laundering behaviors. Future work could therefore integrate dedicated temporal representation learning modules (e.g., dynamic or continuous-time GNNs) to model event-level temporal evolution more explicitly and capture finer-grained laundering sequences. Second, our domain-aware augmentation strategies, while effective, are designed specifically for cryptocurrency AML and may require adaptation for other financial fraud detection scenarios. Developing more generalizable augmentation frameworks that can automatically discover optimal perturbation strategies across different domains remains an open challenge. Third, the current framework treats money laundering detection as a binary classification problem,

but in practice, there exist multiple categories of illicit activities (e.g., ransomware, darknet markets, scams) with distinct behavioral patterns. Extending HeteroGCL to multi-class or hierarchical classification could provide more fine-grained risk assessment. Fourth, our approach relies on centralized graph construction and training, which may raise privacy concerns when dealing with sensitive financial data. Federated learning extensions that enable collaborative model training across multiple institutions without sharing raw transaction data represent a promising direction for sustainable and privacy-preserving AML systems. Fifth, our experimental evaluation is limited to the Elliptic dataset, which contains only Bitcoin transactions and thus reflects a UTXO-based transaction model. Therefore, while HeteroGCL is formulated as a general heterogeneous graph contrastive learning framework, the current empirical results should be interpreted within the scope of this dataset and setting. In particular, we do not claim that the reported scalability and empirical performance directly generalize to other blockchain ecosystems (e.g., account-based platforms such as Ethereum) where smart-contract interactions and different state/account semantics can induce substantially different graph topologies and relational patterns. Evaluating and adapting our graph construction (node/edge types) and augmentation strategies to account-based and smart-contract transaction graphs is an important direction for future work. Finally, while we demonstrate strong predictive performance, enhancing model interpretability through attention visualization and explanation techniques would further increase trust and adoption in regulatory contexts.

Author Contributions: Methodology, J.C. and J.L.; Software, J.C.; Validation, Y.L.; Writing—original draft, J.C., J.L., Y.L., and M.Z.; Writing—review & editing, M.Z.; Visualization, Y.L.; Supervision, Y.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Collins, J. Crypto, crime and control. In *Cryptocurrencies as an Enabler of Organized Crime*; Global Initiative Against Transnational Organized Crime: Geneva, Switzerland, 2022.
2. Foley, S.; Karlsen, J.R.; Putniņš, T.J. Sex, drugs, and bitcoin: How much illegal activity is financed through cryptocurrencies? *Rev. Financ. Stud.* **2019**, *32*, 1798–1853.
3. Fanusie, Y.; Robinson, T. *Bitcoin Laundering: An Analysis of Illicit Flows into Digital Currency Services*; Center on Sanctions and Illicit Finance Memorandum: Washington, DC, USA, 2018.
4. Savage, D.; Wang, Q.; Chou, P.; Zhang, X.; Yu, X. Detection of money laundering groups using supervised learning in networks. *arXiv* **2016**, arXiv:1608.00708. [CrossRef]
5. Weber, M.; Domeniconi, G.; Chen, J.; Weidele, D.K.I.; Bellei, C.; Robinson, T.; Leiserson, C.E. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. *arXiv* **2019**, arXiv:1908.02591. [CrossRef]
6. Alarab, I.; Prakoonwit, S.; Nacer, M.I. Competence of graph convolutional networks for anti-money laundering in bitcoin blockchain. In Proceedings of the 2020 5th International Conference on Machine Learning Technologies, Virtual, 19–21 June 2020; pp. 23–27.
7. Lorenz, J.; Silva, M.I.; Aparício, D.; Ascensão, J.T.; Bizarro, P. Machine learning methods to detect money laundering in the bitcoin blockchain in the presence of label scarcity. In Proceedings of the First ACM International Conference on AI in Finance, New York, NY, USA, 15–16 October 2020; pp. 1–8.
8. Liu, L.; Li, X.; Lan, T.; Cheng, Y.; Chen, W.; Li, Z.; Cao, S.; Han, W.; Zhang, X.; Chai, H. A Survey on Anti-Money Laundering Techniques in Blockchain Systems. *Strateg. Study Chin. Acad. Eng.* **2025**, *27*, 287–303.
9. Wang, D.; Lin, J.; Cui, P.; Jia, Q.; Wang, Z.; Fang, Y.; Yu, Q.; Zhou, J.; Yang, S.; Qi, Y. A semi-supervised graph attentive network for financial fraud detection. In *Proceedings of the 2019 IEEE International Conference on Data Mining (ICDM), Beijing, China, 8–11 November 2019*; IEEE: New York, NY, USA, 2019; pp. 598–607.

10. Hamilton, W.; Ying, Z.; Leskovec, J. Inductive representation learning on large graphs. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 1025–1035.
11. Ying, R.; He, R.; Chen, K.; Eksombatchai, P.; Hamilton, W.L.; Leskovec, J. Graph convolutional neural networks for web-scale recommender systems. In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, London, UK, 19–23 August 2018; pp. 974–983.
12. Liu, Y.; Ao, X.; Qin, Z.; Chi, J.; Feng, J.; Yang, H.; He, Q. Pick and choose: A GNN-based imbalanced learning approach for fraud detection. In Proceedings of the Web Conference, Ljubljana, Slovenia, 19–23 April 2021; pp. 3168–3177.
13. Wang, X.; Ji, H.; Shi, C.; Wang, B.; Ye, Y.; Cui, P.; Yu, P.S. Heterogeneous graph attention network. In Proceedings of the World Wide Web Conference, San Francisco, CA, USA, 13–17 May 2019; pp. 2022–2032.
14. Hu, Z.; Dong, Y.; Wang, K.; Sun, Y. Heterogeneous graph transformer. In Proceedings of the Web Conference, Taipei, Taiwan, 20–24 April 2020; pp. 2704–2710.
15. Chen, T.; Kornblith, S.; Norouzi, M.; Hinton, G. A simple framework for contrastive learning of visual representations. In *Proceedings of the International Conference on Machine Learning, Vienna, Austria, 12–18 July 2020*; PMLR: Cambridge, MA, USA, 2020; pp. 1597–1607.
16. He, K.; Fan, H.; Wu, Y.; Xie, S.; Girshick, R. Momentum contrast for unsupervised visual representation learning. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Seattle, WA, USA, 14–19 June 2020; pp. 9729–9738.
17. Gao, T.; Yao, X.; Chen, D. Simcse: Simple contrastive learning of sentence embeddings. *arXiv* **2021**, arXiv:2104.08821.
18. You, Y.; Chen, T.; Sui, Y.; Chen, T.; Wang, Z.; Shen, Y. Graph contrastive learning with augmentations. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 5812–5823.
19. Zhu, Y.; Xu, Y.; Yu, F.; Liu, Q.; Wu, S.; Wang, L. Deep graph contrastive representation learning. *arXiv* **2020**, arXiv:2006.04131. [CrossRef]
20. Hassani, K.; Khasahmadi, A.H. Contrastive multi-view representation learning on graphs. In *Proceedings of the International Conference on Machine Learning, Vienna, Austria, 12–18 July 2020*; PMLR: Cambridge, MA, USA, 2020; pp. 4116–4126.
21. Oord, A.V.D.; Li, Y.; Vinyals, O. Representation learning with contrastive predictive coding. *arXiv* **2018**, arXiv:1807.03748.
22. Chen, M.; Huang, C.; Xia, L.; Wei, W.; Xu, Y.; Luo, R. Heterogeneous graph contrastive learning for recommendation. In Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining, Virtual, 27 February–3 March 2023; pp. 544–552.
23. Alsaibai, H.; Waheed, S.; Alaali, F.; Wadi, R.A. Online fraud and money laundry in E-Commerce. In *Proceedings of the ECCWS 2020 20th European Conference on Cyber Warfare and Security, Online, 24–25 June 2021*; Academic Conferences and Publishing Limited: Cambridge, MA, USA, 2020; p. 13
24. Colladon, A.F.; Remondi, E. Using social network analysis to prevent money laundering. *Expert Syst. Appl.* **2017**, *67*, 49–58. [CrossRef]
25. Jullum, M.; Løl, A.; Huseby, R.B.; Ånonsen, G.; Lorentzen, J. Detecting money laundering transactions with machine learning. *J. Money Laund. Control.* **2020**, *23*, 173–186. [CrossRef]
26. Pourhabibi, T.; Ong, K.L.; Kam, B.H.; Boo, Y.L. Fraud detection: A systematic literature review of graph-based anomaly detection approaches. *Decis. Support Syst.* **2020**, *133*, 113303. [CrossRef]
27. Bartoletti, M.; Carta, S.; Cimoli, T.; Saia, R. Dissecting Ponzi schemes on Ethereum: Identification, analysis, and impact. *Future Gener. Comput. Syst.* **2020**, *102*, 259–277. [CrossRef]
28. Ostapowicz, M.; Żbikowski, K. Detecting fraudulent accounts on blockchain: A supervised approach. In *Proceedings of the International Conference on Web Information Systems Engineering, Hong Kong, China, 19–22 January 2019*; Springer International Publishing: Cham, Switzerland, 2019; pp. 18–31.
29. Kipf, T.N. Semi-supervised classification with graph convolutional networks. *arXiv* **2016**, arXiv:1609.02907.
30. Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Lio, P.; Bengio, Y. Graph attention networks. *arXiv* **2017**, arXiv:1710.10903.
31. Cheng, D.; Wang, X.; Zhang, Y.; Zhang, L. Graph neural network for fraud detection via spatial-temporal attention. *IEEE Trans. Knowl. Data Eng.* **2020**, *34*, 3800–3813. [CrossRef]
32. Pareja, A.; Domeniconi, G.; Chen, J.; Ma, T.; Suzumura, T.; Kanezashi, H.; Kaler, T.; Schardl, T.; Leiserson, C. Evolvegcn: Evolving graph convolutional networks for dynamic graphs. In Proceedings of the AAAI Conference on Artificial Intelligence, New York, NY, USA, 7–12 February 2020; Volume 34, pp. 5363–5370.
33. Zhang, C.; Song, D.; Huang, C.; Swami, A.; Chawla, N.V. Heterogeneous graph neural network. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Anchorage, AK, USA, 4–8 August 2019; pp. 793–803.
34. Tan, Y.; Wu, B.; Cao, J.; Jiang, B. LLaMA-UTP: Knowledge-guided expert mixture for analyzing uncertain tax positions. *IEEE Access* **2025**, *13*, 90637–90650. [CrossRef]
35. Jiang, B.; Wu, B.; Cao, J.; Tan, Y. Interpretable Fair Value Hierarchy Classification via Hybrid Transformer-GNN Architecture. *IEEE Access* **2025**, *13*, 198142–198163. [CrossRef]

36. Qiu, J.; Chen, Q.; Dong, Y.; Zhang, J.; Yang, H.; Ding, M.; Wang, K.; Tang, J. Gcc: Graph contrastive coding for graph neural network pre-training. In Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Virtual, 6–10 July 2020; pp. 1150–1160.
37. Xu, D.; Cheng, W.; Luo, D.; Chen, H.; Zhang, X. Infogcl: Information-aware graph contrastive learning. *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 30414–30425.
38. Suresh, S.; Li, P.; Hao, C.; Neville, J. Adversarial graph augmentation to improve graph contrastive learning. *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 15920–15933.
39. Wang, X.; Liu, N.; Han, H.; Shi, C. Self-supervised heterogeneous graph neural network with co-contrastive learning. In Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, Virtual, 14–18 August 2021; pp. 1726–1736.